

Shivani.S-192471030

### Q1. Minimum Number of Perfect Squares

Given a number  $n$ , find the minimum number of perfect square numbers whose sum equals  $n$ . The goal is to represent  $n$  using minimum squares. Efficient combination selection is required. Use Dynamic Programming.

Input Format:

$n$  = integer

Sample Input:

$n$  = 12

Expected Output:

Min Count = 3

Ans:

# Q66. Minimum Number of Perfect Squares

# Dynamic Programming

```
def min_perfect_squares(n):
```

```
    dp = [0] + [float("inf")] * n
```

```
    for i in range(1, n + 1):
```

```
        j = 1
```

```
        while j * j <= i:
```

```
            square = j * j
```

```
            dp[i] = min(dp[i], dp[i - square] + 1)
```

```
            j += 1
```

```
    return dp[n]
```

# Sample Input

$n$  = 12

# Output

```
answer = min_perfect_squares(n)
```

```
print ("Min Count =", answer)
```

## Q2. Maximum Length of Pair Chain

Given pairs of numbers, form a chain such that for each pair (a, b), the next pair (c, d) satisfies  $b < c$ . The objective is to maximize the chain length. Use Greedy or Dynamic Programming to solve efficiently.

Input Format:

n = number of pairs

pairs[][]

Sample Input:

n = 3

(1,2) (3,4) (2,3)

Expected Output:

Max Chain = 2

Ans:

# Q67. Maximum Length of Pair Chain

# Greedy Approach

```
def max_pair_chain(pairs):
```

```
    # Sort pairs by their ending value
```

```
    pairs.sort(key=lambda x: x[1])
```

```
    chain_length = 0
```

```
    current_end = float("-inf")
```

```
    for pair in pairs:
```

```
        start = pair[0]
```

```
        end = pair[1]
```

```
        if start > current_end:
```

```
            chain_length += 1
```

```
current_end = end
```

```
return chain_length
```

```
# Sample Input
```

```
n = 3
```

```
pairs = [(1, 2), (3, 4), (2, 3)]
```

```
# Function Call
```

```
answer = max_pair_chain(pairs)
```

```
# Output
```

```
print("Max Chain =", answer)
```